

Resenha - Design by Contract

Nome: Eric Gregório Leal

Curso: Engenharia de Software

Design by Contract

O Design by Contract é uma metodologia para o desenvolvimento de software que propõe que a interação entre diferentes partes de um sistema deve ser regida por contratos formais. Esses contratos definem obrigações e benefícios mútuos entre um "cliente" (quem chama uma rotina) e um "fornecedor" (a rotina chamada). A ideia central é que, se o cliente cumprir certas pré-condições, o fornecedor garante a entrega de um resultado específico, definido por pós-condições. Essa abordagem visa aumentar a confiabilidade do software, tornando as expectativas de cada componente explícitas e verificáveis.

Pré-condições, Pós-condições e Invariantes

O contrato de uma rotina é composto por três elementos lógicos fundamentais. As pré-condições estabelecem os requisitos que devem ser verdadeiros antes da chamada da rotina, sendo de responsabilidade do cliente garanti-las. As pós-condições descrevem o estado que o sistema deve apresentar após a execução, representando o benefício entregue pelo fornecedor. Além disso, existem as invariantes de classe, que são condições que devem permanecer verdadeiras durante toda a vida de uma instância da classe, definindo o que é um estado "saudável" para aquele objeto. Juntos, esses elementos formam uma especificação rigorosa que guia tanto a implementação quanto o teste do sistema.

Tratamento de Exceções

Uma das maiores contribuições do Design by Contract é o seu impacto na gestão de falhas. No modelo de contratos, uma exceção não é apenas um erro genérico, mas sim uma violação de contrato: ou o cliente falhou em satisfazer a pré-condição, ou o fornecedor não conseguiu cumprir sua pós-condição apesar das pré-condições estarem corretas. Isso simplifica o tratamento de erros, pois elimina a necessidade de verificações redundantes de parâmetros (programação defensiva excessiva) dentro das rotinas, já que o contrato estabelece quem é o responsável por cada verificação. Se um contrato é quebrado, o sistema deve interromper a execução ou tentar uma estratégia de recuperação que restabeleça as invariantes.

Herança e Ligação Dinâmica

O artigo também discute como o Design by Contract interage com os pilares da orientação a objetos, como a herança. Para garantir a substituição segura de

classes (Princípio de Liskov), o Design by Contract impõe que uma subclasse pode apenas enfraquecer as pré-condições e fortalecer as pós-condições de um método herdado. Isso significa que um descendente deve aceitar pelo menos tudo o que seu ancestral aceitava e entregar pelo menos tudo o que o ancestral entregava. Essa regra garante que o polimorfismo não quebre a lógica do sistema, mantendo a integridade dos contratos originais mesmo em estruturas de classes complexas.

Conclusão

Dessa forma, a análise de Meyer estabelece que o sucesso de um software confiável depende da clareza dos acordos entre seus componentes. O progresso real na construção de sistemas robustos não vem apenas do tratamento reativo de erros, mas da definição preventiva de responsabilidades através de contratos estáveis. A ideia central é que o design deve ser um exercício de documentar decisões de contratação, garantindo que cada parte do código saiba exatamente o que esperar e o que entregar. Um exemplo prático de uso no mundo real é um sistema de processamento de pagamentos, onde o contrato exige que o valor seja positivo (pré-condição) para garantir que o saldo final seja atualizado corretamente (pós-condição). No fim, a maior dificuldade será mudar a cultura de desenvolvimento para que os contratos sejam vistos como parte integrante do código e não apenas como documentação externa.